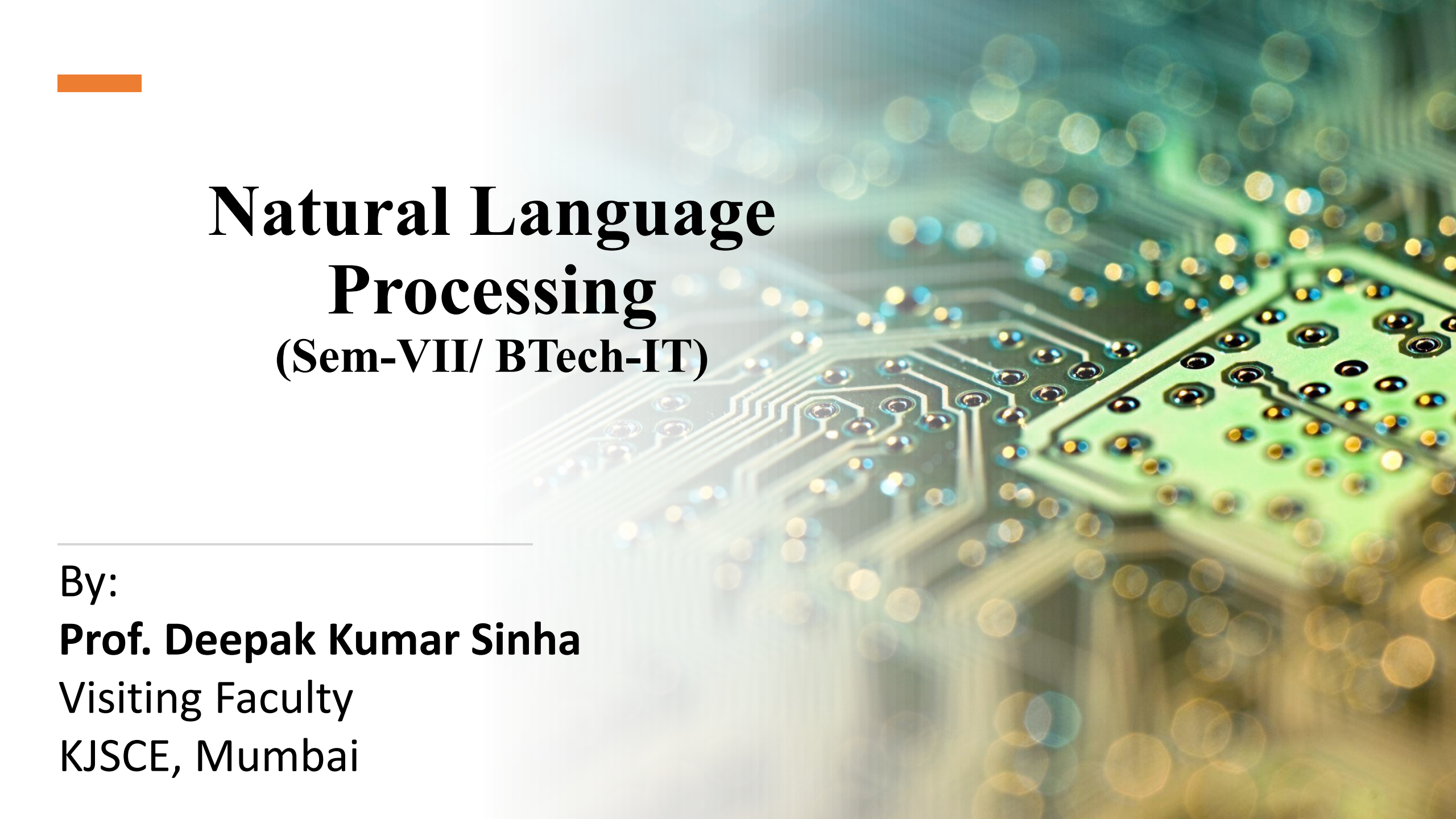
Natural Language Processing

(Sem-VII/ BTech-IT)



By:

Prof. Deepak Kumar Sinha

Visiting Faculty

KJSCE, Mumbai

Unit-2-Words and Word Forms

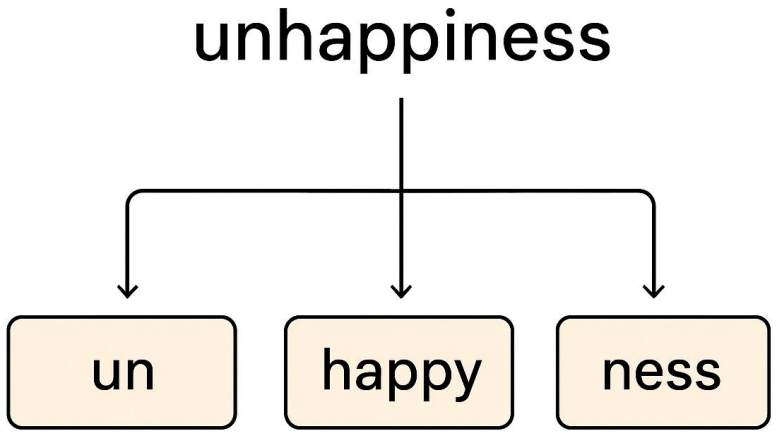
2	**Words and Word Forms**		**10**	**CO2**
	2.1	Morphology fundamentals; Morphological Diversity of IndianLanguages		
	2.2	Morphology Paradigms, Finite State Machine Based Morphology, Automatic Morphology Learning		
	2.3	Shallow Parsing, Named Entities, Maximum Entropy Models,Random Fields		

Morphology Fundamentals

- Morphology
  - Structure of a word as a composition of morphemes
  - Related to word formation rules
  - Functions
    - Inflection
    - Derivation
    - Composition
- Result of morphologic analysis
  - Morphosyntactic categorization (POS)
    - e.g. Parole tagset (VMIP1S0), more than 150 categories for Spanish
    - e.g. Penn Treebank tagset (VBD), about 30 categories for English
  - Morphological features
    - Number, case, gender, lexical functions

Morphology Fundamentals

WHAT IS MORPHOLOGY?



- Each of these parts is called a **morpheme** – the ***smallest unit*** of language that carries meaning.
- The word ***"unhappiness"*** is formed by combining ***three morphemes***, each contributing to the final meaning: "the state of not being happy."
- This is exactly ***what morphology studies***: ***how words are built*** from these meaningful building blocks. In ***computational morphology***, our goal is to ***teach machines*** to perform this analysis automatically.

Core Functions of Morphology

Inflection

Derivation

Composition (Compounding)

1. Inflection

This involves ***changing word forms to express grammatical relationships*** without changing the basic meaning or part of speech.

Let us understand few examples:

Number: house → houses, child → children

Tense: walk → walked → walking

Case (in languages like Hindi): लड़का (ladka) → लड़के (ladke) → लड़कों (ladkon)

English has only 8 inflectional morphemes

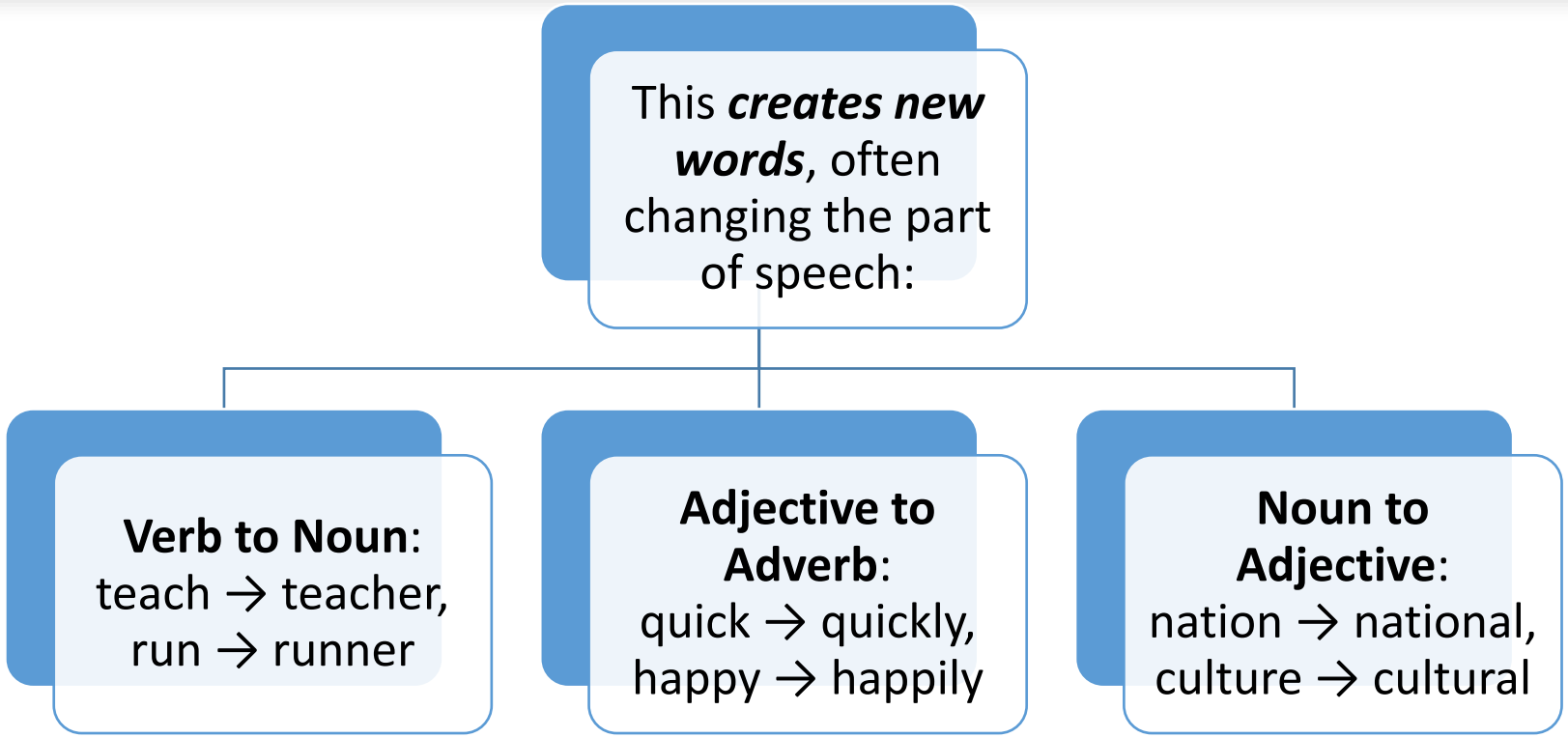
- **-s (plural):** cats
- **-'s (possessive):** cat's
- **-ed (past tense):** walked
- **-en (past participle):** taken
- **-ing (progressive):** walking
- **-s (3rd person singular):** walks
- **-er (comparative):** taller
- **-est (superlative):** tallest

Morphology Fundamentals

Inflectional Morphology

- number
  - house houses
  - cheval chevaux
  - casa casas
- verbal form
  - walk walkes walked walking
  - amo amas aman ...
- gender
  - niño niña

2. Derivation



Morphology Fundamentals

Derivational Morphology

- Form
  - Without change — barcelonés
  - Prefix — inevitable
  - Suffix — importantísimo
- Source
  - verb => adjective — tardar => tardío
  - verb => noun — sufrir => sufrimiento
  - noun => noun — actor => actorazo
  - noun => adjective — atleta => atlético
  - adjective => adjective — rojo => rojizo
  - adjective => adverb — alegre => alegremente

3. Composition (Compounding)

This ***combines multiple roots*** to ***create new words***:

- **English**: blackboard, keyboard, software
- **German**: Computerwissenschaft (computer science)
- **Sanskrit**: राजपुत्र (rajputra = raja + putra = king's son)

Morphology Fundamentals

Morphemes

- 1 morpheme:

  Evitar (verb to avoid)

- 2 morphemes:
  - evitable = evitar + able (adj: can be avoided)
- 3 morphemes:
  - inevitable = in + evitar + able

    (adj: cannot be avoided)
- 4 morphemes:
  - inevitabilidad = in + evitar + able + idad

    (noun: cannot be avoided)

Types of Morphemes

Understanding morphemes is crucial for computational analysis:

Free Morphemes: Can stand alone as words

- Cat, happy, run, book

Bound Morphemes: Must attach to other morphemes

- Prefixes: un-, re-, pre-
- Suffixes: -ing, -ness, -ed
- Infixes: (rare in English, common in some languages)

Morphology Fundamentals

- Morphologic analysis
  - Decompose a word into a concatenation of morphemes
  - Usually some of the morphemes contain the meaning
    - One (root or stem) in flexion and derivation
    - More than one in composition
  - The other (affixes) provide morphological features
- Problems
  - Phonological alterations in morpheme concatenation
  - Morphotactics
    - Which morphemes can be concatenated with which others

Morphology Fundamentals

- Problems
  - Affixes
    - Suffixes, prefixes, infixes, interfixes
  - Inflectional affixes ≠ derivational affixes
  - Derivation implies sometimes a semantic change not always predictible
    - Meaning extensions
    - Lexical rules
  - A derivativational suffix can be followed by an inflectional one
    - love => lover => lovers
  - Inflection does not change POS, sometimes derivation does
  - Inflection affects other words in the sentence
    - agreement

Morphology Fundamentals

- Morphotactics
  - Word formation rules
  - Valid combinations between morphemes
    - Simple concatenation
    - Complex models root/pattern
    - Language dependency regularity
- Phonological alterations (Morphophonology)
  - Changes when concatenating morphemes
  - Source: Phonology, morphology, orthography
  - variable in number and complexity
  - e.g. vocalic harmony

Morphological Analysis: The Computational Challenge

Challenge 1: Phonological Alterations

Words change when morphemes combine:

- beauty + ful → beautiful (y changes to i)
- happy + ness → happiness (y changes to i)

Challenge 3: Ambiguity

Consider "flies":

- fly + s (noun plural: "the insects")
- fly + s (verb third person: "he flies")

Challenge 2: Morphotactics

Not all combinations are valid:

- un + happy + ness ✓ (unhappiness)
- happy + un + ness ✗ (impossible combination)

Practical Examples in Different Complexity Levels

English: cat, run, big

Hindi: घर (ghar - house)

English: cats (cat + s), running (run + ing)

Hindi: घरों (gharon - houses)

English: unhappy (un + happy), quickly (quick + ly)

Spanish: inevitable (in + evit + able)

Spanish: inevitabilidad (in + evit + able + idad)
Turkish: evlerimizden (ev + ler + imiz + den = "from our houses")

Morphological Analysis Techniques

Computational Approaches

1. Rule-Based Approaches

- Use handcrafted linguistic rules
- High accuracy for well-understood languages
- Example: Porter Stemmer for English

Rule: If word ends in -ing and length > 3, remove –ing

"running" → "runn" → "run" (after additional rules)

Computational Approaches

2. Statistical Approaches

- Learn patterns from large datasets
- Use probabilistic models (HMMs, CRFs)
- Better coverage but may lack linguistic insight

3. Neural Network Approaches

- Use deep learning (RNNs, Transformers)
- State-of-the-art results
- Can capture complex patterns but less interpretable

Word Formation Processes

1. Affixation

- **Prefixation**: Adding prefixes (un- + happy = unhappy)
- **Suffixation**: Adding suffixes (happy + -ness = happiness)
- **Infixation**: Inserting within the root (less common in English)
- **Circumfixation**: Adding elements around the root (em- + bold + -en = embolden)

Word Formation Processes

2. Compounding
Combining two or more free morphemes:

- *blackboard*, *keyboard*, *sunrise*
- *ice cream*, *post office* (may be written as separate words)

3. Conversion (Zero Derivation)
Changing word class without adding affixes:

- *email* (noun) → *email* (verb): "I'll email you"
- *google* (noun) → *google* (verb): "Just google it"

Word Formation Processes

4. Blending

Combining parts of two words:

- *smoke + fog = smog*
- *breakfast + lunch = brunch*
- *motor + hotel = motel*

5. Clipping

Shortening existing words:

- *advertisement → ad*
- *telephone → phone*
- *laboratory → lab*

Word Formation Processes

6. Acronymy

Using initial letters:
Light Amplification by Stimulated Emission of Radiation → *laser*
Self Contained Underwater Breathing Apparatus → *scuba*

7. Back-formation

Creating words by removing supposed affixes:
editor → *edit* (historically, "edit" came from "editor")
burglar → *burgle*

Morphological Analysis Techniques

1. Computational Approaches

1. Rule-Based Approaches

- Use handcrafted linguistic rules
- High accuracy for well-understood languages
- Example: Porter Stemmer for English

```
Rule: If word ends in -ing and length > 3, remove -ing
"running" → "runn" → "run" (after additional rules)
```

Morphological Analysis Techniques

1. Computational Approaches

2. Statistical Approaches

- Learn patterns from large datasets
- Use probabilistic models (HMMs, CRFs)
- Better coverage but may lack linguistic insight

3. Neural Network Approaches

- Use deep learning (RNNs, Transformers)
- State-of-the-art results
- Can capture complex patterns but less interpretable

Morphological Analysis Techniques

2. Stemming vs. Lemmatization

Stemming: Crude heuristic process that removes word endings to reach an approximate root form.

Example using Porter Stemmer:

- *jumped* → *jump*
- *friends* → *friend*
- *football* → *footbal* (note the error)
- *mysteries* → *mysteri*/(note the error)

Morphological Analysis Techniques

2. Stemming vs. Lemmatization

Lemmatization: More sophisticated process that uses vocabulary and morphological analysis to return the dictionary form (lemma).

Examples:

- *jumped* → *jump*
- *friends* → *friend*
- *football* → *football* (preserved correctly)
- *mysteries* → *mystery* (correct base form)
- *better* → *good* (irregular comparison)
- *went* → *go* (irregular past tense)

Morphological Analysis Techniques

Differences

Aspect	Stemming	Lemmatization
Method	Rule-based chopping	Vocabulary + morphological analysis
Output	Stem (may not be a real word)	Lemma (dictionary word)
Speed	Fast	Slower (requires POS tagging)
Accuracy	Lower	Higher
Context Awareness	No	Yes (considers POS)

Morphological Diversity of Indian Languages

Overview of Indian Linguistic Landscape

India presents one of the world's most complex morphological landscapes with:

- **4 major language families**: Indo-Aryan, Dravidian, Austro-Asiatic, Sino-Tibetan
- **22 official languages** plus hundreds of regional varieties
- **Diverse morphological types**: From relatively isolating to highly agglutinative

Key Characteristics of Indian Languages:

1. **Rich morphological systems** (compared to English)
2. **Free word order** (morphology indicates relationships)
3. **Complex verb systems** with rich TAM (Tense-Aspect-Modality) marking
4. **Agglutinative tendencies** (especially Dravidian languages)

Morphological Diversity of Indian Languages

Indo-Aryan Languages

Indo-Aryan languages show **fusional morphology** with significant inflectional complexity.

Noun Inflection:

Base: *लड़का* (ladka - boy)

Case	Singular	Plural
Direct	लड़का (ladka)	लड़के (ladke)
Oblique	लड़के (ladke)	लड़कों (ladkon)

With Postpositions:

- *लड़के को* (ladke ko - to the boy, accusative)
- *लड़कों में* (ladkon mein - among the boys, locative)

Morphological Diversity of Indian Languages

Verb Conjugation:

Base: जा (jaa - go)

Present Tense:

- मैं जाता हूँ (main jaata hun - I go, masculine)
- मैं जाती हूँ (main jaati hun - I go, feminine)
- वे जाते हैं (ve jaate hain - they go, masculine)

Complex Word Formation:

- अध्यापिका (adhyaapika) = अध्याप (adhyaap, "teach") + इका (ika, feminine suffix)
- अध्यापिकाओं (adhyaapikaon) = अध्यापिका + ओं (plural oblique)

Morphological Diversity of Indian Languages

Bengali Morphology

Bengali Morphology:

Verb Conjugation Complexity:

Base: *কর* (kar - do)

- *করি* (kori - I do)
- *করছি* (korchi - I am doing, continuous)
- *করেছি* (korechi - I have done, perfect)
- *করতাম* (kortam - I used to do, habitual past)

Computational Challenges

Challenges in Processing Indian Languages:

- **Rich Morphology**: Single words can express what English needs entire phrases for
- **Free Word Order**: Grammatical relations encoded in morphology, not position
- **Multiple Scripts**: देवनागरी, தமிழ், తెలుగు, ಕನ್ನಡ, മലയാളം
- **Sandhi Rules**: Sound changes at morpheme boundaries
- **Code-mixing**: Multiple languages in single texts

Computational Challenges

Example of Computational Complexity:

Hindi: *मैंने उससे कहा था कि वह जल्दी आए।*

(maine usse kaha tha ki vah jaldi aaye)

Morphological Analysis needed:

- *मैंने*(maine) = मैं (main, "I") + ने (ne, ergative marker)
- *उससे*(usse) = उस (us, "that") + से (se, ablative/instrumental)
- *कहा*(kaha) = कह (kah, "say") + आ (aa, past tense marker)
- *था*(tha) = auxiliary agreeing with subject
- *आए*(aaye) = आ (aa, "come") + ए (e, subjunctive)

Practical Applications and Tools

(Morphological Analyzers)

Definition: Tools that perform syntactic analysis of words and identify their root forms and morphological features.

Popular Tools:

- **For Hindi:**
  - **Hindi Shallow Parser** (IIIT Hyderabad)
  - **Hindi Morphological Analyzer** (CDAC)
- **For Dravidian Languages:**
  - **Tamil Morphological Analyzer** (AU-KBC Research Centre)
  - **Telugu Morphological Analyzer** (IIIT Hyderabad)
- **Multilingual:**
  - **spaCy** (supports multiple Indian languages)
  - **Polyglot**
  - **UDPipe**

Practical Applications and Tools

(Morphological Analyzers)

Example Usage:

```
Input: "लड़कों"
Output:
- Root: "लड़का" (boy)
- Number: Plural
- Case: Oblique
- Gender: Masculine
```

Real-world Applications

1. Machine Translation:

- **Challenge**: Hindi "मैंने पुस्तक पढ़ी है" → English "I have read the book"
- **Need**: Recognize that मैंने indicates perfect aspect, पुस्तक is direct object, पढ़ी shows feminine agreement

2. Information Retrieval:

- **Search Query**: "teaching" should match documents with "teach," "teacher," "taught"
- **Indian Language Example**: Searching for "पढ़ना" should match "पढ़ता," "पढ़ाई," "पाठशाला"

3. Sentiment Analysis:

- **Challenge**: "अच्छा नहीं" (accha nahi - not good) vs "बुरा" (bura - bad)
- **Need**: Understand negation patterns and morphological modifications

4. Text Summarization:

- **Challenge**: Recognize that "गायक," "गायिका," and "गान" all relate to singing
- **Benefit**: Better content understanding for summary generation

What are Morphological Paradigms?

- A **morphological paradigm** is a complete set of related word forms associated with a given lexeme. Simply put, it's a systematic arrangement of all the different forms a word can take.
- Lets understand with an example. Consider the English verb "walk":

walk, walks, walked, walking

- These four forms constitute the **inflectional paradigm** of the verb "walk". Each form serves a different grammatical function - present tense, third person singular, past tense, and present participle respectively.

Types of Paradigms

1. Inflectional Paradigms

These involve adding affixes that don't change
the basic meaning or part of speech of the word.

Noun Paradigm Example:

- Stem: "cat"
- Singular: cat
- Plural: cats
- Possessive singular: cat's
- Possessive plural: cats'

Verb Paradigm Example:

- Stem: "sing"
- Present: sing/sings
- Past: sang
- Past participle: sung
- Present participle: singing

Types of Paradigms

2. Derivational Paradigms

- These create new words with different meanings or parts of speech.
- **Example with stem "head":**

  ahead, behead, header, headlong, headship, heady, subhead

- Notice how each derived word has a different meaning and sometimes a different part of speech.

Paradigmatic Relationships

- Understanding paradigms helps us identify **paradigmatic similarity** - how different word classes behave similarly. For instance, all English regular verbs follow the pattern:

  Base form → Base + "s" (3rd person) → Base + "ed" (past) → Base + "ing" (progressive)

- This predictable pattern allows NLP systems to generalize morphological rules across similar word classes.

Computational Significance

From a computational perspective, paradigms are crucial because they:

- **Reduce memory requirements** - instead of storing every word form, we store rules
- **Enable prediction** - if we know one form, we can generate others
- **Handle unknown words** - we can apply paradigmatic patterns to new vocabulary

Real-world Application:
Google Search uses paradigmatic knowledge. When you search for "running shoes," it knows to also consider results about "run," "runs," and "ran" because these belong to the same paradigm.

Finite State Machine Based Morphology

Introduction to Finite State Machines

- **Finite State Machines (FSMs)** provide an elegant mathematical framework for processing morphology.
- A **Finite State Automaton (FSA)** consists of:
  - ✓ A finite set of states Q = {q_0, q_1, ..., q_n}
  - ✓ A finite alphabet Σ of input symbols
  - ✓ A start state q_0
  - ✓ A set of final states F
  - ✓ A transition function δ

Two-Level Morphology

The breakthrough came with **Kimmo Koskenniemi's Two-Level Morphology** in the 1980s. This system uses **two levels**:

- **Lexical Level**: The underlying morphological representation
- **Surface Level**: The actual word as it appears in text

Example:

- Lexical: cat+N+Pl
- Surface: cats

The "+" symbols are morpheme boundaries that don't appear in the surface form.

Finite State Transducers (FSTs)

FSTs extend FSAs by having both input and output tapes. They can transform one string into another, making them perfect for morphological analysis.

FST Components:

- ✓ Input alphabet Σ (surface forms)
- ✓ Output alphabet Δ (lexical forms)
- ✓ Transition function mapping input:output pairs
- ✓ Output function generating morphological tags

Practical Example - English Plural Formation

- Lets understand how an FST would handle "foxes" → "fox+N+Pl“

  State 1: f:f → State 1
  State 1: o:o → State 1
  State 1: x:x → State 1
  State 1: e:ε → State 2 (delete 'e')
  State 2: s:+N+Pl → State 3 (final)

- Here, ε represents an empty symbol, and the FST correctly identifies that "foxes" should be analyzed as "fox" + plural marker.

Morphotactics

- **Morphotactics** defines the permissible combinations of morphemes. For example, in English:

Prefix + Root + Suffix is valid: "un-break-able"

****Root + Prefix + Suffix is invalid: "*break-un-able"***

- FSTs can encode these constraints elegantly through state transitions.

Spelling Rules

- Real morphology involves **spelling changes**. Consider:
  - try + ing → trying (y→i change)
  - take + ing → taking (e deletion)
  - run + ing → running (consonant doubling)
- FSTs handle these through **orthographic rules** that modify the surface representation.
- **FST for E-Deletion Rule:**

State 1 --e:ε--> State 2 --s:s--> State 3 (accepts "cakes" → "cake+s")
State 1 --other:other--> State 1 (non-e characters pass through)

Advantages of FST-Based Morphology

- **Bidirectional**: Same FST can analyze (cats → cat+N+Pl) and generate (cat+N+Pl → cats)
- **Efficient**: Linear time complexity for processing
- **Compositional**: Multiple FSTs can be combined for complex morphological phenomena
- **Language-independent framework**: The same mathematical model works across languages

Commercial Applications

- Spell checkers
- Machine translation systems
- Information retrieval
- Grammar checkers

Automatic Morphology Learning (AML)

- We have already seen how words can be broken down into smaller meaningful units called **morphemes**.

- For example:

  'unhappiness' → 'un-' (prefix) + 'happy' (stem) + '-ness' (suffix)

- The challenge in NLP is: *How can a **machine automatically learn** these structures without being explicitly told?*

- This is where **Automatic Morphology Learning (AML)** comes in. It tries to identify stems, affixes, and morphological rules directly from large text corpora."

AML-Problem & Approaches

The problem is simple to state but hard to solve

- **Input**: A large corpus of words.
- **Output**: Automatic identification of stems, affixes, and morphological rules.

There are two main approaches used to handle this:

1. No prior morphological knowledge (unsupervised learning).

- The system starts with raw words and tries to discover patterns.
- Example: Goldsmith (2001), Brent (1999), Snover & Brent (2001, 2002).

2. Using prior knowledge (semi-supervised or rule-based help).

- Here, linguists may provide some basic rules, or the system starts with known irregulars.
- Example: Oliver et al. (2002).

How does AML work ?

1. Identification of morpheme boundaries

- Classic work by Zellig Harris used ***conditional entropy of prefixes and suffixes.***
- **Idea:** If a certain prefix/suffix makes word endings ***very predictable***, it's likely a true morpheme.
- **Example:**
  - In English, suffix ***'-ed' has high predictability***: 'walked', 'played', 'jumped'.
  - In contrast, the sequence '-lp' rarely occurs, so it's not a valid suffix.

2. Statistical methods with bigrams and trigrams

- Systems look at ***frequent co-occurring patterns***.
- If ***'ing' often*** follows stems, the system marks it as a ***possible suffix***.

How does AML work ?

3. Learning patterns or rules	Systems try to ***map pairs*** of words:
	‘run’ → ‘running’, ‘swim’ → ‘swimming’
	From such mappings, they infer the suffix rule: ‘add –ing’.
4. Global (top-down) approaches	Goldsmith, Brent, and de Marcken proposed global statistical approaches:
	Instead of word-by-word guessing, they ***look for the best explanation*** of all words in the corpus together.”

Goldsmith's MDL Approach

The idea of MDL: ***The best morphological analysis is the one that explains the words with the shortest description possible.***

1. Initial Partition: Split each word into {stem + suffix}.

- Example: 'teacher' → 'teach' + 'er'.
- But not all splits are correct: 'butter' ≠ 'butt' + 'er'.

2. Split-all-words strategy

- Candidate stem-suffix splits are checked across many words.
- If a suffix is valid, it should appear across multiple words.

Example:

- 'teach-er', 'sing-er', 'work-er'.
- The suffix '-er' is valid because it repeats in different contexts.

Goldsmith's MDL Approach

3. Mutual Information (MI) strategy

Looks at ***how strongly*** a stem is associated with a suffix.

If the suffix ***appears in many contexts with different stems***, it's more likely ***real.***

4. Learning Signatures

A 'signature' is a ***set of suffixes*** that can attach to the same stem.

Example: For the stem 'walk': {walk, walked, walking, walker}.

Signature = {Ø, -ed, -ing, -er}.

By ***compressing the data into signatures***, MDL ensures efficient learning."

Semi-Automatic Morphological Analysis

- Oliver (2004) proposed a **semi-automatic method**:
    - Starts with a set of **manually written morphological rules**.
    - ***Uses a format TL:TF:Desc*** → Lemma ending, Form ending, POS.
    - ***Lists irregular and closed-class words*** (like ‘is’, ‘the’, ‘and’).
- For example:
    - **Rule:** Verb stem ending in ‘e’ + ‘ing’ → drop ‘e’ + ‘ing’.
    - **Lemma:** ‘make’ → Form: ‘making’.
- This approach was ***tested on large corpora***:
    - Serbo-Croatian (9 million words).
    - Russian (16 million words).

So, semi-automatic methods can ***scale better because they start with some linguistic knowledge*** instead of reinventing everything.”

Shallow Parsing

Shallow parsing, also known as ***light parsing or chunking***, is a natural language processing (NLP) technique focused on ***identifying and extracting key informational units or keywords from sentences*** without performing a full grammatical analysis.

Unlike ***deep parsing***, which creates a complete parse tree to capture the ***entire sentence structure***, ***shallow parsing*** concentrates on recognizing ***broader linguistic components*** such as ***noun phrases, verb phrases, and prepositional phrases***.

The ***main objective*** of shallow parsing is to ***balance computational efficiency*** with linguistic accuracy. Rather than exploring the complex syntactic connections within a sentence, it ***aims to extract the most significant elements*** of the sentence structure for use in further NLP tasks.

Purpose of Shallow Parsing

Information Extraction: Shallow parsing helps in ***extracting key elements*** from the text, such as ***named entities, noun phrases, and verb phrases,*** which are crucial for tasks like entity recognition, relationship extraction, and summarization.

Text Understanding: By identifying ***important phrase fragments***, shallow parsing aids in ***improving the analysis*** and understanding of text, enhancing its syntactic and semantic interpretation.

Computational Efficiency: Compared to deep parsing, shallow parsing techniques are generally ***more computationally efficient***, making them ideal for processing large amounts of text in real-time or near-real-time applications.

Feature Engineering: For machine learning models involved in tasks such as text classification, sentiment analysis, and information retrieval, ***shallow parsing provides useful features*** by capturing higher-level textual structures.

Shallow Parsing- Syntax & Structure

- **Meaning**: Syntax refers to the arrangement of words and phrases to form coherent sentences. Shallow parsing ***analyzes syntax to identify important segments*** such as nouns, verbs, and prepositional phrases. These ***segments represent the core structural components*** of a sentence.
- **Structure**: The ***output*** of shallow parsing generally ***reveals a sentence's hierarchical structure***, with words organized into phrases. While the specific method used for shallow parsing can affect this structure, it typically ***identifies the relationships between words and phrases without delving into deeper*** semantic analysis.

Shallow Parsing- Syntax & Structure

Example:

- **Sentence**: "The cat sat on the mat."
- **Shallow Parsing Output**:
  - ***Noun Phrase (NP):*** "The cat"
  - ***Verb Phrase (VP):*** "sat"
  - ***Prepositional Phrase (PP):*** "on the mat"

Linguistic Units of Shallow Parsing

- **Words**: Each word in the sentence is evaluated for ***its function*** within a phrase and its ***grammatical category*** (e.g., part of speech).
- **Phrases**: Phrases are ***groups of words that function as a unit*** within the sentence. Examples include noun phrases (NP), verb phrases (VP), and prepositional phrases (PP).
- **Dependencies**: Shallow parsing can also ***identify dependencies between words*** or phrases, such as subject-verb or verb-object relationships.
- **Named Entities**: In some cases, shallow parsing algorithms ***can identify named entities***, such as the names of people, organizations, or locations.

Common Techniques

- **Part-of-Speech (POS) Tagging**: This technique involves ***classifying the words in a sentence into grammatical categories***, such as nouns, verbs, adjectives, etc. These categories, or tags, help determine the syntactic roles of words in a sentence.
- **Chunking**: Chunking is the process of ***grouping consecutive words in a sentence*** into meaningful chunks or phrases, such as noun phrases (NP) or verb phrases (VP). This technique often ***relies on POS tagging*** to define the boundaries of these chunks.
- **Named Entity Recognition (NER)**: ***NER identifies and categorizes named entities in text***, such as names of places, organizations, and individuals. It extracts significant pieces of information from text, making it a form of shallow parsing.
- **Regular Expressions**: Regular expressions are patterns ***used to recognize specific linguistic structures in text***. These patterns can be applied to extract words or chunks based on predefined criteria.
- **Statistical Models**: Statistical models, such as Hidden Markov Models (HMMs) and Conditional Random Fields (CRFs), can be ***trained on annotated corpora to predict sentence structures and identify important chunks or phrases***.

Applications of Shallow Parsing

- **Information Extraction**
  Information extraction ***involves locating and obtaining specific details*** from unstructured text. Shallow parsing ***helps identify key entities and relationships in text***, such as names, dates, and events, by detecting syntactic patterns and phrases.
- **Question Answering Systems**
  Shallow parsing plays a vital role in question-answering systems by ***identifying relevant phrases and entities in both questions and passages***. It helps break down the ***grammatical structure of queries, highlighting subjects, objects, and other key components***. This enhances the system's ability to retrieve ***accurate answers from large text corpora***, benefiting sectors like information retrieval, customer service, and education.
- **Sentiment Analysis**
  Sentiment analysis focuses on ***identifying the sentiment expressed in a text whether positive, negative, or neutral***. Shallow parsing ***aids in recognizing sentiment-laden words*** and phrases, such as noun phrases, adjectives, and adverbs. By analyzing sentence structure, shallow parsing enables the extraction of sentiment-bearing phrases, allowing sentiment analysis algorithms to categorize the tone of a document, review, or social media post. This has wide applications in market research, customer feedback, brand monitoring, and reputation management.

Applications of Shallow Parsing

- **Machine Translation**
  Shallow parsing ***enhances machine translation*** by ***identifying syntactic units and sentence fragments*** that ***retain their meaning during translation***. It ensures that the ***syntactic structure*** and relationships between words ***are preserved***, enabling translations that are more accurate and contextually coherent.
- **Text Summarization**
  Text summarization involves ***condensing lengthy documents into concise, informative summaries***. Shallow parsing algorithms help identify important keywords, phrases, and sections within the text, facilitating the extraction of key information and removal of irrelevant content. This aids in creating clear and meaningful summaries for applications like content creation, news aggregation, and document summarization.

What is named entity recognition (NER)?

- Named Entity Recognition (NER) in NLP focuses on ***identifying and categorizing important information known as entities*** in text.
- These entities can be ***names of people, places, organizations, dates***, etc.
- It helps in ***transforming unstructured text into structured information*** which helps in tasks like text summarization, knowledge graph creation and question answering.

NER-Example

Consider this news excerpt:

"Barack Obama, the former president, visited Google headquarters in Mountain View, California, on January 15th, 2023, to discuss the $2.5 billion investment in artificial intelligence research."

An NER system would identify:

- ***Person: "Barack Obama"***
- ***Organization: "Google"***
- ***Location: "Mountain View, California"***
- ***Date: "January 15th, 2023"***
- ***Money: "$2.5 billion"***

This extraction transforms unstructured text into structured data that can be stored in databases, used for information retrieval, or fed into downstream applications.

Named Entity Recognition Methods

Rule-based Methods

- Rule-based methods are grounded in ***manually crafted rules***. They identify and classify named entities based on linguistic patterns, regular expressions, or dictionaries.
- While they shine in specific domains where entities are well-defined, such as extracting standard medical terms from clinical notes, their scalability is limited. They might struggle with large or diverse datasets due to the rigidity of predefined rules.

Statistical Methods

- Transitioning from manual rules, statistical methods ***employ models like Hidden Markov Models (HMM) or Conditional Random Fields (CRF).***
- They predict named entities based on likelihoods derived from training data. These methods are apt for tasks with ample labeled datasets at their disposal. Their strength lies in generalizing across diverse texts, but they're only as good as the training data they're fed.

Named Entity Recognition Methods

Machine Learning Methods

- Machine learning methods take it a step further by using algorithms such as ***decision trees or support vector machines***. They ***learn from labeled data*** to predict named entities. Their widespread adoption in modern NER systems is attributed to their prowess in handling vast datasets and intricate patterns. However, they're hungry for substantial labeled data and can be computationally demanding.

Deep Learning Methods

- The latest in the line are deep learning methods, which harness the ***power of neural networks.*** Recurrent Neural Networks (RNN) and transformers have become the go-to for many due to their ability to model long-term dependencies in text. They're ideal for large-scale tasks with abundant training data but come with the caveat of requiring significant computational might.

Named Entity Recognition Methods

Hybrid Methods

- Lastly, there's no one-size-fits-all in NER, leading to the emergence of hybrid methods. These techniques ***intertwine rule-based, statistical, and machine learning approaches, aiming to capture the best of all worlds.*** They're especially valuable when extracting entities from diverse sources, offering the flexibility of multiple methods. However, their intertwined nature can make them complex to implement and maintain.

Named Entity Recognition Use Cases

News aggregation. NER is instrumental in categorizing news articles by the primary entities mentioned. This categorization aids readers in swiftly locating stories about specific people, places, or organizations, streamlining the news consumption process.

Customer support. Analyzing customer queries becomes more efficient with NER. Companies can swiftly pinpoint common issues related to specific products or services, ensuring that customer concerns are addressed promptly and effectively.

Research. For academics and researchers, NER is a boon. It allows them to scan vast volumes of text, identifying mentions of specific entities relevant to their studies. This automated extraction speeds up the research process and ensures comprehensive data analysis.

Legal document analysis. In the legal sector, sifting through lengthy documents to find relevant entities like names, dates, or locations can be tedious. NER automates this, making legal research and analysis more efficient.

Working of Named Entity Recognition (NER)

Analyzing the Text: It processes entire text to locate words or phrases that could represent entities.

Finding Sentence Boundaries: It identifies starting and ending of sentences using punctuation and capitalization which helps in maintaining meaning and context of entities.

Tokenizing and Part-of-Speech Tagging: Text is broken into tokens (words) and each token is tagged with its grammatical role which provides important clues for identifying entities.

Entity Detection and Classification: Tokens or groups of tokens that match patterns of known entities are recognized and classified into predefined categories like Person, Organization, Location etc.

Model Training and Refinement: Machine learning models are trained using labeled datasets and they improve over time by learning patterns and relationships between words.

Adapting to New Contexts: A well-trained model can generalize to different languages, styles and unseen types of entities by learning from context.

Understanding the Maximum Entropy

Sentence: 'I am going to the bank.'

Ambiguity: Bank = financial institution OR river side

Features: Previous word = 'the', current word = 'bank'.

MaxEnt uses all available clues without bias.

Philosophy of MaxEntropy

01

Among all models that fit data, choose the one with maximum entropy.

02

Entropy = uncertainty; maximizing it prevents adding bias.

03

Analogy: If you don’t know about someone, don’t assume extra things.

Entropy Calculation

We want to estimate:

$$P(y \mid x) = \frac{1}{Z(x)} \exp\left(\sum_i \lambda_i f_i(x, y)\right)$$

- $y \rightarrow$ the class label (e.g., POS tag, sentiment)
- $x \rightarrow$ the context/features (e.g., words, prefixes, capitalization)
- $f_i(x, y) \rightarrow$ feature functions (binary indicators: does feature i apply for input x and output y?)
- $\lambda_i \rightarrow$ weights for features, learned from data
- $Z(x) \rightarrow$ normalization factor (to make probabilities sum to 1)

This is basically a **log-linear model**.

The model assigns higher probability to outcomes that strongly match features with large positive weights.

Example

Suppose we want to classify a word as **Verb (V)** or **Noun (N)**.

Features:

1. If word ends with "-ing" → feature for Verb
2. If previous word is "the" → feature for Noun

For input: *'the building'*

- Feature 1 (-ing) fires → suggests Verb
- Feature 2 (previous word = the) fires → suggests Noun

The model combines both signals with their weights and decides the most probable class.

Notice: **Unlike Naive Bayes, MaxEnt doesn't assume features are independent.** It just lets them all contribute."

Comparison with Naive Bayes

Naive Bayes: assumes independence of features.

MaxEnt: no independence assumption.

Naive Bayes: multiplicative probabilities.

MaxEnt: log-linear probabilities.

MaxEnt = more flexible, better for NLP.

Applications in NLP

Part-of-Speech Tagging: Predicting tags using features like suffix, capitalization, surrounding words.

Named Entity Recognition: Features like word shape (‘New York’ capitalized), previous tag, context.

Sentiment Analysis: Features like presence of words (‘great’, ‘terrible’).

Word Sense Disambiguation: Features like surrounding words (our ‘bank’ example).

Limitations

- Training can be computationally expensive.
- Requires careful feature engineering.
- Best results with lots of data + strong features.

Random Fields

- A **random field** is fundamentally a mathematical framework for representing the joint probability distribution over a set of random variables that are spatially or temporally related.
- Think of it as a sophisticated way to model how different pieces of information influence each other.
- In formal terms, a random field **F** is a collection of random variables $\{F_t : t \in T\}$ where each F_t is a random variable indexed by elements in some space T.
- For NLP, this space T typically represents positions in a text sequence.

The Evolution: From Independence to Dependence

- Traditional approaches in NLP often assume **independence** between words or labels.
- For example, in a simple bag-of-words model, each word is treated independently. But language doesn't work this way.
- Consider these examples:
    - "President Obama" - knowing that "President" appears makes "Obama" more likely to be a person's name
    - "New York" - seeing "New" followed by "York" should be recognized as a single location entity
    - "Running shoes" vs. "Running water" - the same word "running" has different meanings based on context
- This is where random fields become essential—they explicitly model these **dependencies** between neighboring elements.

Types of Random Fields in NLP

Markov Random Fields (MRFs): These model joint probability distributions over undirected graphs. Each variable depends on its immediate neighbors according to the Markov property.

Conditional Random Fields (CRFs): These are discriminative models that directly model the conditional probability P(Y|X) where Y is our output sequence (like POS tags) and X is our input sequence (like words).

The Graph Structure

- Random fields are represented as graphs where:
  - ✓ **Nodes (vertices)** represent random variables (words, labels, features)
  - ✓ **Edges** represent dependencies between variables
  - ✓ **Cliques** are fully connected subsets of nodes that define local relationships
- For a linear sequence like a sentence, we typically use a **linear chain** structure where each position connects only to its immediate neighbors.

Markov Random Fields

An MRF is:

- An **undirected graph** where each node is a random variable.
- Dependencies are captured through **cliques** (fully connected subsets of nodes).

The joint probability is given by:

$$P(X) = \frac{1}{Z} \prod_{C \in cliques} \psi_C(X_C)$$

- X → all variables (like word tags).
- C → a clique in the graph.
- ψ_C → a potential function for that clique.
- Z → partition function (normalizing constant).

The idea: Instead of assigning probabilities directly, we assign **scores** (potentials), then normalize.

Example

- Suppose we want to label a sentence with POS tags.
  - ✓ Word Ravi could be NOUN.
  - ✓ Word plays could be VERB.
  - ✓ But the probability of Ravi=VERB is low if plays is right after.
- So, we build a graph where each node is a word's tag, and edges connect neighboring words. The MRF captures these **dependencies**.
- **But one issue:** MRFs model joint probability of both input (words) and output (tags). This makes them hard to use when input is observed and only output needs prediction."

Conditional Random Fields (CRFs)

- This is where **Conditional Random Fields (CRFs)** come in as one of the most important Random Field models in NLP.

- Instead of modeling joint probability $P(X, Y)$, CRFs directly model **conditional probability**:

$$P(Y \mid X)$$

  - $X \rightarrow$ observed data (sentence words).
  - $Y \rightarrow$ hidden structure we want (tags, labels)

- This is much more practical for NLP because we always know the words (input), and we want to predict the tags/labels (output).

Conditional Random Fields (CRFs)

$$P(Y|X) = \frac{1}{Z(X)} \exp\left(\sum_i \sum_k \lambda_k f_k(y_{i-1}, y_i, X, i)\right)$$

- f_k → feature functions (like "Is the word capitalized?", "Is the previous tag a noun?").
- λ_k → weights learned from training data.
- $Z(X)$ → normalizing factor.